



TEORÍA DE ALGORITMOS
(TB024) CURSO 02 - BUCHWALD

Trabajo Práctico 1

Los Algoritmos Greedy son juegos de niños

[illegible]

17 de septiembre de 2026

Juán David Chaparro
111481

Nicolás Llosas
105397

Joaquín Acevedo
89863

1. Análisis del problema

Se dispone una fila de n monedas de valores $M_1, M_2, \dots, M_n$, dados en ese orden. En cada turno sólo puede retirarse la primera o la última moneda de la fila. Sophia comienza y, debido a que Mateo es demasiado pequeño para decidir, Sophia controla tanto sus propias elecciones como las elecciones asignadas a Mateo. Gana quien acumula una suma mayor.

Hipótesis consideradas

- Hay una fila de n monedas con valores conocidos, con $n \geq 1$ arbitrario: no suponemos ninguna paridad en particular.
- Los valores de las monedas son positivos.
- En cada turno solamente se puede elegir uno de los dos extremos.
- Sophia controla tanto sus elecciones como las de Mateo.
- Sophia siempre empieza, para sí misma.
- El orden de los valores en la fila es arbitrario: no suponemos que estén ordenados ni que sigan ningún patrón.

Deliberadamente no incluimos entre las hipótesis que los valores sean distintos entre sí, aunque el enunciado hable de una fila de monedas “de diferentes valores”. La razón es que los casos de prueba provistos por la cátedra no cumplen esa condición: en `20000.txt` los 20 000 valores toman solamente 991 valores distintos, todos en el rango $[10, 1000]$, con lo cual cada valor se repite unas veinte veces en promedio. Un resultado que dependiera de la unicidad de los valores no cubriría los propios casos con los que se evalúa el trabajo.

Por eso demostramos primero el resultado general, válido sin ninguna hipótesis sobre los valores, y luego lo reforzamos bajo la hipótesis de valores distintos. La salvedad del enunciado sobre una cantidad par de monedas de mismo valor, en cuyo caso siempre sería empate más allá de la estrategia de Sophia, corresponde justamente al caso en que esa hipótesis se relaja; lo analizamos en detalle en la sección de variabilidad de los valores.

Qué entendemos por solución óptima

Es necesario precisar la noción de optimalidad, porque hay dos lecturas posibles y no son equivalentes:

- **Garantizar la victoria:** lograr que la suma de Sophia sea mayor que la de Mateo en toda instancia válida.
- **Maximizar la suma de Sophia:** lograr que su suma sea la mayor alcanzable entre todas las secuencias de elecciones posibles.

Adoptamos la primera, que es la que pide el enunciado: el objetivo no es simplemente “ganar mucho”, sino elegir toda la secuencia de monedas de forma que Sophia termine con una suma mayor. La distinción no es retórica: mostramos con un contraejemplo que nuestro algoritmo resuelve el primer problema y no el segundo, de modo que son objetivos genuinamente distintos y el algoritmo cumple uno solo de los dos.

Resolución

2. Elección de monedas

El problema es el siguiente: dados los n valores de todas las monedas, indicar qué monedas debe ir eligiendo Sophia para sí misma y para Mateo, de tal forma que se asegure de ganar siempre.

Presentamos una regla Greedy que resuelve el problema, la demostración de que siempre gana, su complejidad y su verificación empírica. Al final documentamos otras reglas que consideramos y descartamos, porque el motivo por el que fallan ilustra qué propiedad sostiene la correctitud de la que sí funciona.

2.1. La regla Greedy

La regla es una única decisión que se repite, tomada por *ronda* (un turno de Sophia seguido del turno que ella asigna a Mateo):

De las dos monedas que están en los extremos, Sophia toma la de mayor valor para sí misma; a continuación, de las dos que quedan en los extremos, le asigna a Mateo la de menor valor.

2.2. ¿Por qué el algoritmo es Greedy?

No alcanza con afirmar que el algoritmo toma decisiones locales esperando llegar a un óptimo global. Hacen falta dos cosas: poder enunciar la regla Greedy, y poder enunciar el *óptimo local* al que esa regla lleva, local al momento actual y relacionado con la métrica del óptimo global.

La regla es la enunciada arriba, y se aplica sobre el estado actual: lo único que lee son los valores de los dos extremos de la fila vigente. No consulta el arreglo original, no precalcula nada al inicio y no vuelve sobre decisiones ya tomadas. La distinción importa: una regla que ordenara las monedas al comienzo, o que decidiera en función de propiedades calculadas sobre la fila completa antes de empezar, estaría mirando el estado inicial y no el actual, y por lo tanto no sería Greedy.

El óptimo local es, en cada ronda, la diferencia que esa ronda aporta al marcador:

$$d_i = S_i - M_i.$$

En el turno de Sophia la regla maximiza su ganancia inmediata entre las dos opciones disponibles; en el turno de Mateo minimiza la de él entre las dos que quedan. Ambas decisiones se toman mirando exclusivamente el estado vigente.

La relación entre el óptimo local y el global es, en este problema, inusualmente directa: la métrica global se *descompone aditivamente* en las locales. En efecto,

$$\text{ganancia}_{\text{Sophia}} - \text{ganancia}_{\text{Mateo}} = \sum_i d_i,$$

de modo que el objetivo global se consigue garantizando el signo de cada aporte local d_i . No hace falta que los aportes se compensen ni interactúen entre sí: se suman. Eso es exactamente lo que demuestra el lema de la sección siguiente, y es la razón de fondo por la que un enfoque Greedy funciona en este problema.

2.3. Solución óptima

Para la demostración se agrupan los movimientos en rondas de dos jugadas: primero la jugada de Sophia y luego la jugada asignada a Mateo. Denotaremos por S_i el valor obtenido por Sophia en la ronda i y por M_i el valor recibido por Mateo en esa misma ronda.

Lema local

Sea una ronda que comienza con extremos L y R , con al menos dos monedas en la fila. Sin pérdida de generalidad supongamos $L \geq R$; el caso $R > L$ es simétrico, intercambiando el papel de los dos extremos.

La regla hace que Sophia tome L . Después de retirar L , R continúa siendo un extremo y aparece un nuevo extremo X . Como a Mateo se le asigna el menor de ambos,

$$M = \min(X, R).$$

Por definición $M \leq R$, y como $L \geq R$ se obtiene, encadenando,

$$M = \min(X, R) \leq R \leq L.$$

Por lo tanto, en cada ronda completa Sophia obtiene un valor mayor o igual que el de Mateo. Si además $L \neq R$, entonces $L > R$ y la última desigualdad se vuelve estricta:

$$M \leq R < L.$$

Teorema de victoria

Agrupando la ejecución en pares de movimientos

$$(S_1, M_1), (S_2, M_2), \dots,$$

el lema garantiza que $S_i \geq M_i$ para cada ronda completa, cualesquiera sean los valores de las monedas y su orden.

Sumando las desigualdades resulta

$$\sum_i S_i \geq \sum_i M_i,$$

es decir que Sophia nunca pierde. Este resultado no usa ninguna hipótesis sobre los valores.

Si además los valores son distintos entre sí, los dos extremos de cualquier tramo de la fila son distintos, de modo que el lema aplica en su forma estricta en toda ronda y $S_i > M_i$. Sumando,

$$\sum_i S_i > \sum_i M_i$$

y Sophia gana de forma estricta, que es la condición pedida.

Conviene observar que la hipótesis de valores distintos se usa en un único punto de todo el argumento: el paso $L \neq R$. Todo lo demás vale sin ella.

Caso par e impar

Si n es par, todas las monedas se reparten en $n/2$ rondas completas.

Si n es impar, existen $\lfloor n/2 \rfloor$ rondas completas y queda una última moneda para Sophia. Como los valores son positivos, esa moneda adicional sólo puede aumentar su ventaja; en particular, con n impar el empate es imposible incluso si se admiten valores repetidos.

Qué significa óptimo: ganar no es maximizar

El teorema afirma que Sophia *gana*, no que obtenga el máximo posible. Las dos cosas no coinciden, ni siquiera con valores todos distintos. Consideremos la instancia

$$[10, 2, 5, 20, 50].$$

La regla le asigna a Sophia las monedas 50, 20 y 5, con un total de 75 contra 12 de Mateo: Sophia gana, como asegura el teorema. Sin embargo, la mejor suma que Sophia podría obtener es 80, lograda con la asignación 10, 50, 20. Para alcanzarla debe empezar tomando la moneda de valor 10 en lugar de la de valor 50, es decir renunciar deliberadamente al extremo mayor para no quedar obligada a exponer el 50 en el turno siguiente. La regla Greedy lo prohíbe por construcción.

No se trata de un caso aislado. Mediante una búsqueda exhaustiva sobre instancias de valores distintos encontramos otros ejemplos del mismo fenómeno, como [13, 28, 43, 31, 14, 21], donde la regla obtiene 92 para Sophia mientras que el máximo alcanzable es 95.

Esto confirma que la noción de optimalidad pertinente es la de garantizar la victoria, y que maximizar la suma de Sophia es un problema distinto, para el cual esta regla Greedy no es óptima.

2.4. Algoritmo

A continuación se presenta el algoritmo que implementa la regla. Los índices *primera* y *ultima* delimitan el tramo de la fila que aún queda; como sólo se puede retirar de los extremos, no hace falta borrar elementos de la lista.

```
1 def juego_monedas(monedas):
2     primera = 0
3     ultima = len(monedas) - 1
4
5     ganancia_sophia = 0
6     ganancia_mateo = 0
7     movimientos = []
8     monedas_sophia = []
9     monedas_mateo = []
10
11     turno_sophia = True
12
13     while primera <= ultima:
14         if turno_sophia:
15             if monedas[primera] > monedas[ultima]:
16                 ganancia_sophia += monedas[primera]
17                 monedas_sophia.append(monedas[primera])
18                 movimientos.append("Primera moneda para Sophia")
19                 primera += 1
20             else:
21                 ganancia_sophia += monedas[ultima]
22                 monedas_sophia.append(monedas[ultima])
23                 movimientos.append("Ultima moneda para Sophia")
24                 ultima -= 1
25         else:
26             if monedas[primera] < monedas[ultima]:
27                 ganancia_mateo += monedas[primera]
28                 monedas_mateo.append(monedas[primera])
29                 movimientos.append("Primera moneda para Mateo")
30                 primera += 1
31             else:
32                 ganancia_mateo += monedas[ultima]
33                 monedas_mateo.append(monedas[ultima])
34                 movimientos.append("Ultima moneda para Mateo")
35                 ultima -= 1
36
37         turno_sophia = not turno_sophia
38
39     return ganancia_sophia, ganancia_mateo, movimientos, monedas_sophia,
40           monedas_mateo
```

Una observación sobre los operadores $>$ y $<$: cuando los dos extremos tienen el mismo valor, la condición resulta falsa y se toma el último. Al valer igual, la elección es indistinta para el resultado, pero hay que fijar un criterio para que el algoritmo sea determinista. Elegimos éste porque es el que reproduce exactamente las salidas de referencia provistas por la cátedra.

2.5. Complejidad del algoritmo

Complejidad temporal

El algoritmo recorre implícitamente la fila una sola vez. En cada iteración se elimina conceptualmente una moneda avanzando el índice *primera* o retrocediendo el índice *ultima*, y se realizan solamente operaciones de tiempo constante:

- Acceder a `monedas[primera]` y `monedas[ultima]`.
- Comparar los dos valores.
- Sumar el valor a la ganancia correspondiente.
- Incrementar *primera* o decrementar *ultima*.
- Cambiar de turno.

Todas estas operaciones tienen costo $\mathcal{O}(1)$. Como en cada vuelta se procesa exactamente una moneda y existen n monedas,

$$T(n) = n \cdot \mathcal{O}(1),$$

por lo tanto

$$T(n) = \Theta(n)$$

Usamos Θ y no sólo $\mathcal{O}$ porque la cota es exacta: el algoritmo necesariamente procesa todas las monedas, independientemente de sus valores, de modo que tanto en el mejor como en el peor caso realiza n iteraciones.

Complejidad espacial

Si se excluye la lista de movimientos, que se conserva únicamente para informar la salida, el algoritmo utiliza una cantidad constante de variables auxiliares: índices, acumuladores y algunos valores temporales. El espacio auxiliar es entonces $\mathcal{O}(1)$. Si se almacena explícitamente la secuencia completa de n movimientos, la salida ocupa $\mathcal{O}(n)$.

2.6. Variabilidad de los valores

Efecto sobre el tiempo de ejecución

En el modelo habitual, donde comparar y sumar dos valores cuesta $\mathcal{O}(1)$, la variabilidad de los valores no afecta la complejidad temporal. Las dos entradas

1;1;1;1;1;1

2;999999;3;45000;8;10000000

tienen la misma cantidad de monedas, y el algoritmo realiza la misma cantidad de iteraciones en ambos casos. Los valores sólo determinan qué rama de la comparación se ejecuta, pero ambas ramas realizan una cantidad constante de operaciones. Por lo tanto no importa si los valores son todos iguales, muy diferentes entre sí, están ordenados o desordenados: mientras haya n monedas, el algoritmo realiza n decisiones y

$$T_{\text{mejor}}(n) = T_{\text{promedio}}(n) = T_{\text{peor}}(n) = \Theta(n).$$

Hay, sin embargo, un punto donde la magnitud sí importa, y conviene señalarlo porque contradice la respuesta rápida. El supuesto de que comparar dos valores cuesta $\mathcal{O}(1)$ vale mientras

los valores quepan en una palabra de máquina. Con enteros de precisión arbitraria, como los de Python, comparar y sumar cuesta proporcional a la cantidad de dígitos internos, es decir $\mathcal{O}(\log v)$ para un valor v . Contando bits, el tiempo total es entonces

$$\Theta(n \log v),$$

con v el mayor valor de la instancia. El efecto es medible: en el experimento de la Sección de mediciones, pasar de valores que caben en una palabra a valores de unos 2000 bits encarece la ejecución alrededor de un 30 % con n fijo. Para los tamaños de valores del enunciado el término $\log v$ es constante y la complejidad es $\Theta(n)$, pero la distinción es la respuesta completa a la pregunta.

Efecto sobre la optimalidad

Mientras los valores sean positivos y distintos, la magnitud de las diferencias no afecta la garantía de victoria: la demostración sólo usa relaciones de orden. Una ventaja local de 1 y una de 1 000 000 son igualmente suficientes para establecer $S_i > M_i$. Tampoco influye el orden en que estén dispuestos: el lema se aplica al tramo vigente sin suponer nada sobre él.

Si se admiten valores repetidos, en cambio, desaparecen las desigualdades estrictas y el resultado se debilita a $S \geq M$: Sophia nunca pierde, pero puede empatar. Conviene distinguir dos situaciones, porque sólo una es una limitación del algoritmo:

- **Empates inevitables.** En instancias de la forma $[a, b, b, \dots, b, a]$ con $a < b$ — por ejemplo $[1, 2, 2, 1]$ o $[2, 3, 3, 3, 2]$ — ninguna estrategia permite ganar: el máximo alcanzable por Sophia es exactamente la mitad del total. Aquí la regla hace lo mejor posible y el empate es atribuible a la instancia. Es a estos casos a los que apunta la salvedad del enunciado.
- **Empates evitables.** Existen instancias con repetidos que sí son ganables y en las que, sin embargo, la regla empata. Por ejemplo $[1, 2, 3, 3, 2, 1]$: la regla produce 6 contra 6, mientras que la asignación 1, 3, 3 le daría a Sophia 7 contra 5. Aquí la limitación es del algoritmo y no de la instancia.

Verificamos además que ningún criterio de desempate salva a la regla: probando las cuatro combinaciones posibles (Sophia y Mateo prefiriendo el extremo izquierdo o el derecho ante valores iguales) se obtienen exactamente los mismos fallos. La causa es estructural: en $[1, 2, 3, 3, 2, 1]$ la única forma de ganar exige asignarle a Mateo el extremo *mayor* en alguna ronda, jugada que la regla no puede realizar.

Este punto es relevante para la lectura de los casos de prueba. Las instancias grandes provistas por la cátedra tienen muchos valores repetidos: `20000.txt` contiene 20 000 monedas que toman solamente 991 valores distintos, todos en el rango $[10, 1000]$. Es decir que la hipótesis de valores distintos no se cumple allí, y lo que cubre esos casos es el resultado general $S \geq M$. En la práctica Sophia gana igual y con holgura, como se muestra más abajo.

2.7. Ejemplo de ejecución

Se utiliza la instancia $[2, 100, 1, 3]$.

Turno	Monedas disponibles	Jugador	Elección
1	$[2, 100, 1, 3]$	Sophia	3 derecha
2	$[2, 100, 1]$	Mateo	1 derecha
3	$[2, 100]$	Sophia	100 derecha
4	$[2]$	Mateo	2

Cuadro 1: Monedas disponibles y elecciones realizadas en cada turno

Resultado: Sophia = 103, Mateo = 3. Por lo tanto, Sophia gana.

2.8. Corroboración mediante casos exhaustivos

Como verificación complementaria a la demostración, se ensayaron exhaustivamente todas las permutaciones de los valores $1, \dots, n$ para n entre 2 y 8. Al ser permutaciones, los valores son distintos, de modo que corresponde el caso estricto del teorema. En ninguna de esas entradas el algoritmo produjo empate o derrota de Sophia. Esta comprobación no sustituye la prueba formal, pero sirve como control de implementación y permite detectar errores en condiciones de borde.

n	Cantidad de permutaciones	Empates/derrotas observados
2	2	0
3	6	0
4	24	0
5	120	0
6	720	0
7	5040	0
8	40320	0

Cuadro 2: Cantidad de permutaciones y empates/derrotas observados

2.9. Verificación con los casos provistos por la cátedra

Se ejecutó el algoritmo sobre las siete instancias provistas. Sophia gana en todas, incluso en las dos más grandes, donde la densidad de repeticiones es alta y por lo tanto aplica el resultado general y no el estricto. Las ganancias coinciden con las de referencia en los siete casos.

Caso	n	Sophia	Mateo	Gana Sophia
20.txt	20	7165	3849	sí
25.txt	25	9635	3577	sí
50.txt	50	17750	8414	sí
100.txt	100	35009	14664	sí
1000.txt	1000	357814	148841	sí
10000.txt	10000	3550307	1484321	sí
20000.txt	20000	7139357	2981705	sí

Cuadro 3: Resultados sobre los casos de prueba de la cátedra

2.10. Reglas descartadas

Antes de fijar la regla anterior consideramos otras alternativas. Documentamos dos de ellas, porque el motivo por el que fallan es informativo.

Asignarle a Mateo el extremo opuesto al que eligió Sophia

En lugar de darle a Mateo el menor de los extremos disponibles, se le asigna directamente el extremo que quedó libre tras la jugada de Sophia. Esta regla también garantiza la victoria, porque Sophia se queda con $\max(L, R)$ y Mateo recibe $\min(L, R)$, de modo que el invariante por ronda se preserva. No la elegimos porque es estrictamente peor: al no mirar el extremo que aparece, desperdicia oportunidades de dejarle a Mateo una moneda todavía más chica, y en consecuencia empata en más instancias con valores repetidos que la regla adoptada.

Dejarle a Mateo la peor de sus mejores opciones

Sophia elige el extremo que minimice el valor de la mejor moneda que quedaría expuesta para Mateo, y luego le asigna el menor de los extremos disponibles. La intuición es que conviene

arruinarle las opciones al rival. Sin embargo esta regla **pierde**: en la instancia

[1, 3, 4, 3]

produce 5 para Sophia contra 6 para Mateo.

El motivo del fallo se ve con el criterio de la Sección 2.2. La regla sí usa el estado actual, y su óptimo local se puede enunciar: minimizar la mejor opción que le quedaría a Mateo. Pero ese óptimo local está relacionado sólo con *la mitad* de la métrica global. El marcador depende de la diferencia entre lo que junta cada uno, y esta regla atiende únicamente al término de Mateo, ignorando por completo lo que gana Sophia. Concretamente, puede llevarla a tomar el extremo menor, con lo cual se rompe el paso $L \geq R$ del lema y con él la garantía por ronda.

Esto muestra que el invariante no es un detalle de la redacción de la demostración, sino la propiedad que efectivamente sostiene la correctitud: una regla que no lo preserva no sólo deja de ser óptima, sino que puede perder.

3. Mediciones

Se realizaron tres experimentos, que corresponden a los dos enfoques posibles de la consigna sobre variabilidad de los valores: uno sobre el tamaño de la entrada, uno sobre la varianza de los valores dentro de una misma instancia, y uno sobre la magnitud de los valores entre instancias distintas. El código está en `mediciones.py`.

Metodología

Para cada configuración se ejecutó el algoritmo diez veces y se tomó el *mínimo* de los tiempos, no el promedio. El algoritmo es determinista: dada la misma entrada ejecuta exactamente las mismas operaciones, de modo que toda diferencia entre corridas proviene de interferencia externa, principalmente de la planificación del sistema operativo. Esa interferencia sólo puede agregar tiempo, nunca quitarlo, por lo que la corrida más rápida es la estimación menos contaminada del costo real, mientras que el promedio mezcla la señal con el ruido del entorno. Se descarta además la primera corrida de cada configuración, por el calentamiento de las cachés.

Los tamaños se miden en orden aleatorio y no en orden creciente de n . La máquina sufre variaciones de carga a lo largo de la corrida; si se midiera en orden creciente, esa variación quedaría correlacionada con n y podría confundirse con una tendencia del algoritmo. Al aleatorizar el orden se reparte como ruido no sistemático.

Las instancias del primer experimento se generan como una permutación aleatoria de $1, \dots, n$, de modo que los valores son distintos. Vale señalar una falencia que detectamos en la generación: en una primera versión usábamos `random.sample` sobre un rango mayor, y eso producía un salto reproducible de $1,6\times$ en los tiempos para n entre 90 000 y 100 000, ajeno al algoritmo. La causa es que `random.sample` elige internamente entre dos estrategias según el tamaño de la población, y justo en esa franja copiaba una lista auxiliar de un millón de elementos, lo cual degradaba la localidad de memoria. Es un ejemplo concreto de cómo el método de generación de los casos de prueba puede contaminar la medición de un algoritmo que no tiene nada que ver con el problema.

Experimento A: tiempo en función de la cantidad de monedas

n	Tiempo (s)
1000	0.000094
5000	0.000465
10000	0.000916
25000	0.002398
50000	0.005002
100000	0.010546
250000	0.031229

Cuadro 4: Tiempo de ejecución según la cantidad de monedas

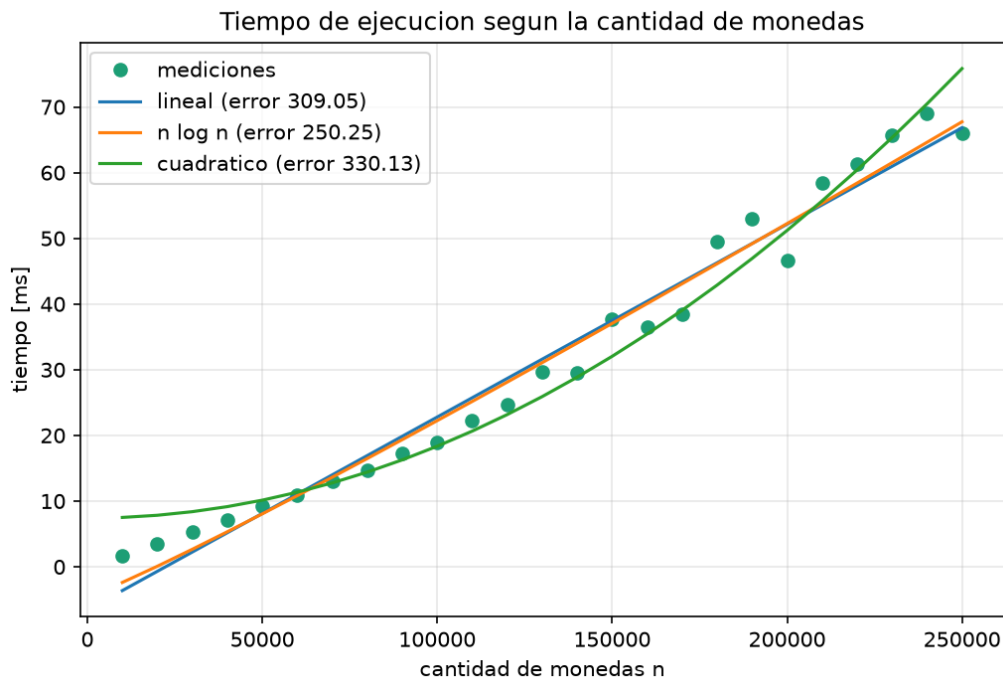


Figura 1: Tiempo de ejecución en función de n , con los tres modelos ajustados por cuadrados mínimos

El ajuste por cuadrados mínimos se realizó armando la matriz A cuyas columnas son el modelo evaluado en cada n y una columna de unos, y resolviendo el sistema normal

$$A^t A \bar{x} = A^t \bar{b},$$

tal como se plantea en el material de la cátedra. El error informado es $\|\bar{b} - A\bar{x}\|^2$, es decir la suma de los cuadrados de las diferencias entre los tiempos medidos y los predichos.

Con el modelo lineal $t(n) = a \cdot n + b$ se obtuvo $a = 1,249 \times 10^{-7}$ s/moneda, $b = -6,348 \times 10^{-4}$ s y $R^2 = 0,9958$.

Sobre la comparación entre modelos conviene ser preciso. El modelo cuadrático queda descartado con claridad: su error es más de un orden de magnitud mayor que el de los otros dos. Separar n de $n \log n$ mediante el error del ajuste, en cambio, es frágil: en el rango medido $\log_2 n$ varía solamente entre 13 y 18, de modo que $n \log n$ se comporta casi como una recta de mayor pendiente y los dos errores quedan del mismo orden, pudiendo alternarse entre corridas. Por eso el ajuste por sí solo no constituye prueba concluyente de linealidad.

El discriminante más robusto es el cociente t/n , que bajo un modelo lineal debería ser constante y bajo $n \log n$ debería crecer como $\log_2 n$. Sobre el rango medido ese cociente crece muy poco, frente al aproximadamente 35 % que exigiría $n \log n$, y eso sí respalda la linealidad. De todos modos, lo que fija la complejidad en $\Theta(n)$ es el argumento teórico de la sección anterior: una vuelta del ciclo por moneda, con trabajo constante en cada vuelta.

Experimento B: varianza de los valores dentro de la instancia

Se fija n y se generan los valores alrededor de un mismo centro, abriendo progresivamente el rango. La media se mantiene constante y sólo cambia la dispersión; el caso de dispersión nula corresponde a todas las monedas de igual valor.

Dispersión	Desvío estándar	Tiempo (ms)
<i>completar con la salida de mediciones.py</i>		

Cuadro 5: Tiempo de ejecución según la varianza de los valores, con n fijo

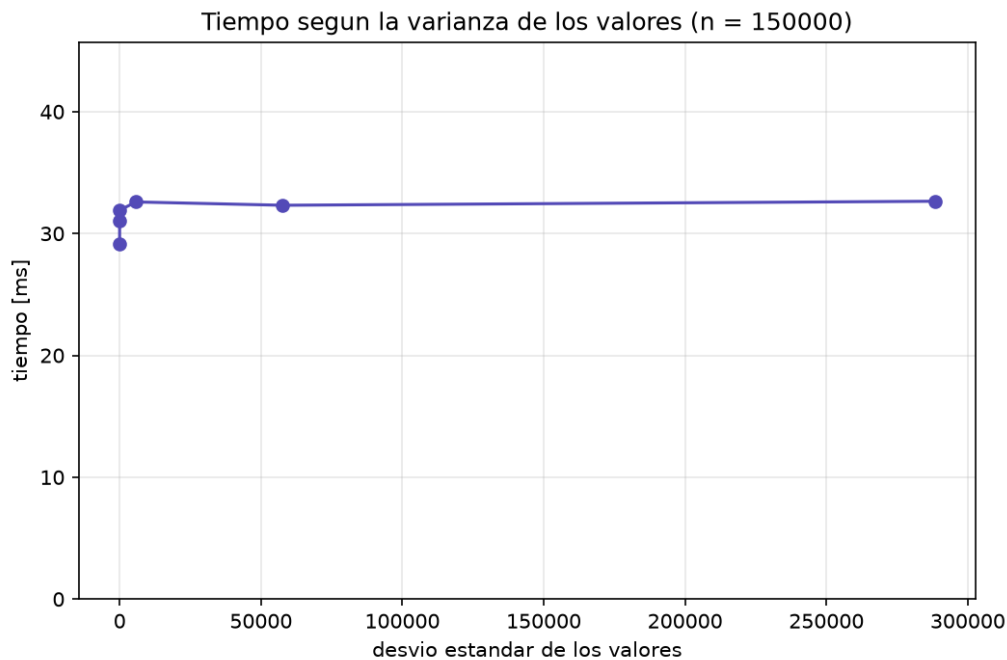


Figura 2: Tiempo de ejecución en función del desvío estándar de los valores, con n fijo

Los tiempos se mantienen constantes dentro del margen de ruido de la medición, sin tendencia apreciable. Esto es lo esperable: la varianza de los valores determina qué rama de la comparación se ejecuta, pero ambas ramas realizan la misma cantidad de operaciones. En particular, el caso degenerado de todas las monedas iguales no es ni más rápido ni más lento que el de máxima dispersión.

Experimento C: magnitud de los valores entre instancias

Se fija n y se generan instancias cuyos valores tienen magnitudes crecientes, desde unidades hasta números de miles de bits.

Magnitud	Tamaño (bits)	Tiempo (ms)
<i>completar con la salida de mediciones.py</i>		

Cuadro 6: Tiempo de ejecución según la magnitud de los valores, con n fijo

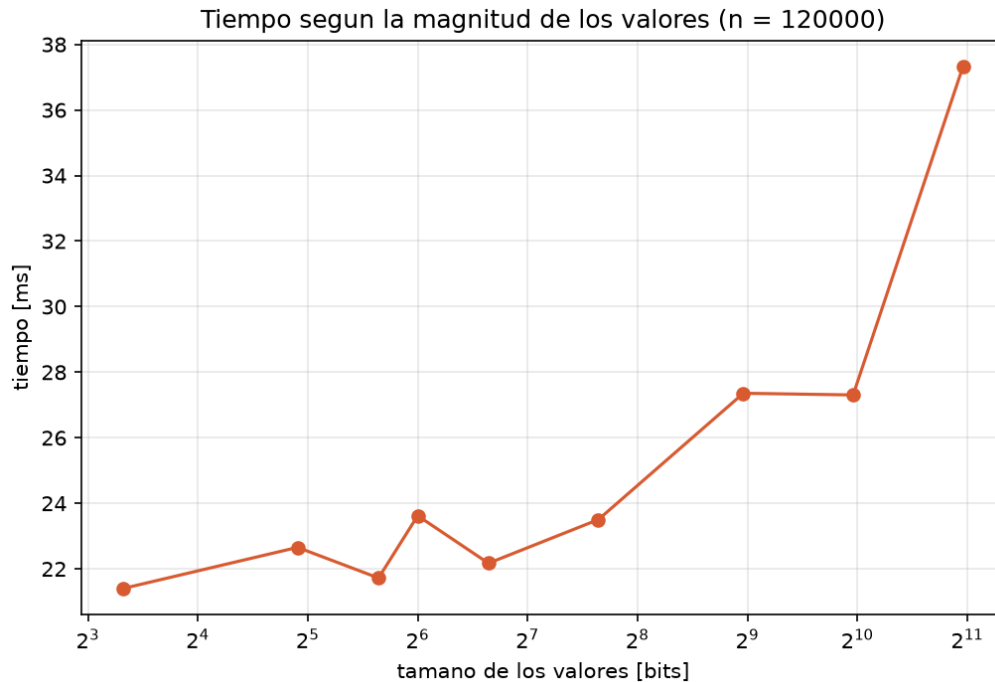


Figura 3: Tiempo de ejecución en función del tamaño en bits de los valores, con n fijo

Este es el único experimento donde la variabilidad de los valores sí afecta el tiempo, y confirma empíricamente la observación teórica de la sección de variabilidad. Mientras los valores caben en una palabra de máquina el tiempo se mantiene plano, porque comparar y sumar son operaciones de costo constante. A partir de cierta magnitud el tiempo empieza a crecer: Python representa los enteros grandes en varios dígitos internos de 30 bits, y comparar o sumar dos enteros cuesta proporcional a la cantidad de dígitos. Contando bits, el tiempo total es entonces $\Theta(n \log v)$ con v el mayor valor de la instancia.

Para los tamaños de valores del enunciado el término $\log v$ es constante y la complejidad efectiva es $\Theta(n)$, pero el experimento muestra que la respuesta “la magnitud de los valores no afecta el tiempo” es incompleta: depende del modelo de costo que se adopte.

4. Conclusiones

El problema admite una solución Greedy sorprendentemente simple, y la razón de fondo es estructural: la métrica global se descompone aditivamente en rondas. Como la diferencia final entre las sumas de Sophia y de Mateo es la suma de las diferencias que aporta cada ronda, basta con garantizar el signo de cada aporte local para garantizar el resultado global. Los aportes no necesitan compensarse ni interactuar entre sí. Esa descomposición es lo que hace que un óptimo local se traduzca directamente en el óptimo global, y es lo que permite una demostración de cuatro líneas en lugar de una recurrencia.

Vale la pena notar cuánto depende esto de que Sophia controle *ambas* elecciones. Si Mateo eligiera por sí mismo, la regla de tomar el extremo mayor dejaría de funcionar: con la instancia $[8, 15, 3, 7]$, Sophia tomaría el 8, Mateo el 15, y Sophia perdería 15 a 18. Un cambio menor en

el enunciado vuelve incorrecto el algoritmo y obliga a programación dinámica. Esto ilustra que la correctitud de un algoritmo Greedy no es una propiedad de la regla en abstracto, sino de la relación entre la regla y la estructura del problema.

La distinción entre garantizar la victoria y maximizar la ganancia resultó ser el punto conceptual más importante del trabajo. Nuestra regla cumple lo primero y no lo segundo: en $[10, 2, 5, 20, 50]$ obtiene 75 cuando el máximo alcanzable es 80. Precisar qué significa “óptimo” antes de demostrar algo evitó demostrar la propiedad equivocada, y el contraejemplo muestra que la diferencia entre ambas lecturas no es una tecnicidad.

El análisis de las reglas descartadas fue más informativo que el de la regla que funciona. La regla de dejarle a Mateo la peor de sus mejores opciones parece razonable y sin embargo pierde en $[1, 3, 4, 3]$. Su óptimo local es enunciable, pero está relacionado con sólo la mitad de la métrica global: atiende a lo que recibe Mateo e ignora lo que gana Sophia. Eso corta la cadena entre lo local y lo global, y es exactamente el tipo de falla que un algoritmo Greedy puede tener sin que se note a simple vista.

Sobre las hipótesis, encontramos una tensión entre el enunciado y los casos de prueba. El enunciado plantea monedas de valores diferentes, pero las instancias grandes provistas tienen 991 valores distintos sobre 20 000 monedas. Eso nos llevó a demostrar el resultado general sin hipótesis sobre los valores y recién después reforzarlo, lo cual además hizo evidente que la unicidad se usa en un único paso de todo el argumento. Al relajarla aparecen dos fenómenos distintos que conviene no confundir: instancias donde el empate es inevitable para cualquier estrategia, como $[1, 2, 2, 1]$, e instancias ganables donde nuestra regla empatiza de todos modos, como $[1, 2, 3, 3, 2, 1]$. Sólo el segundo caso es una limitación del algoritmo, y verificamos que ningún criterio de desempate lo corrige.

En el plano empírico, la lección fue metodológica. Las mediciones descartan con claridad el modelo cuadrático, pero no permiten separar n de $n \log n$: en el rango medido ambos modelos ajustan con error del mismo orden, y cuál gana puede cambiar entre corridas. Afirmar que el ajuste lineal “confirma” la complejidad teórica habría sido incorrecto. El cociente t/n , casi constante, es mucho más informativo que el error del ajuste, y de todos modos la complejidad la fija el argumento teórico y no la medición. También aprendimos que el método de generación de los casos de prueba puede contaminar el resultado: el salto de $1,6\times$ que introducía `random.sample` en cierta franja de tamaños no tenía nada que ver con el algoritmo.

Por último, el experimento sobre la magnitud de los valores dejó el único caso donde la variabilidad sí afecta el tiempo. La respuesta corta a la consigna es que no lo afecta, y es correcta en el modelo de costo uniforme; la respuesta completa es que con enteros de precisión arbitraria el costo por operación crece con la cantidad de dígitos y el tiempo total pasa a ser $\Theta(n \log v)$. Que el resultado dependa del modelo de costo elegido, y no sólo del algoritmo, fue probablemente lo menos obvio de todo el trabajo.